

Rule 1

Rule 2

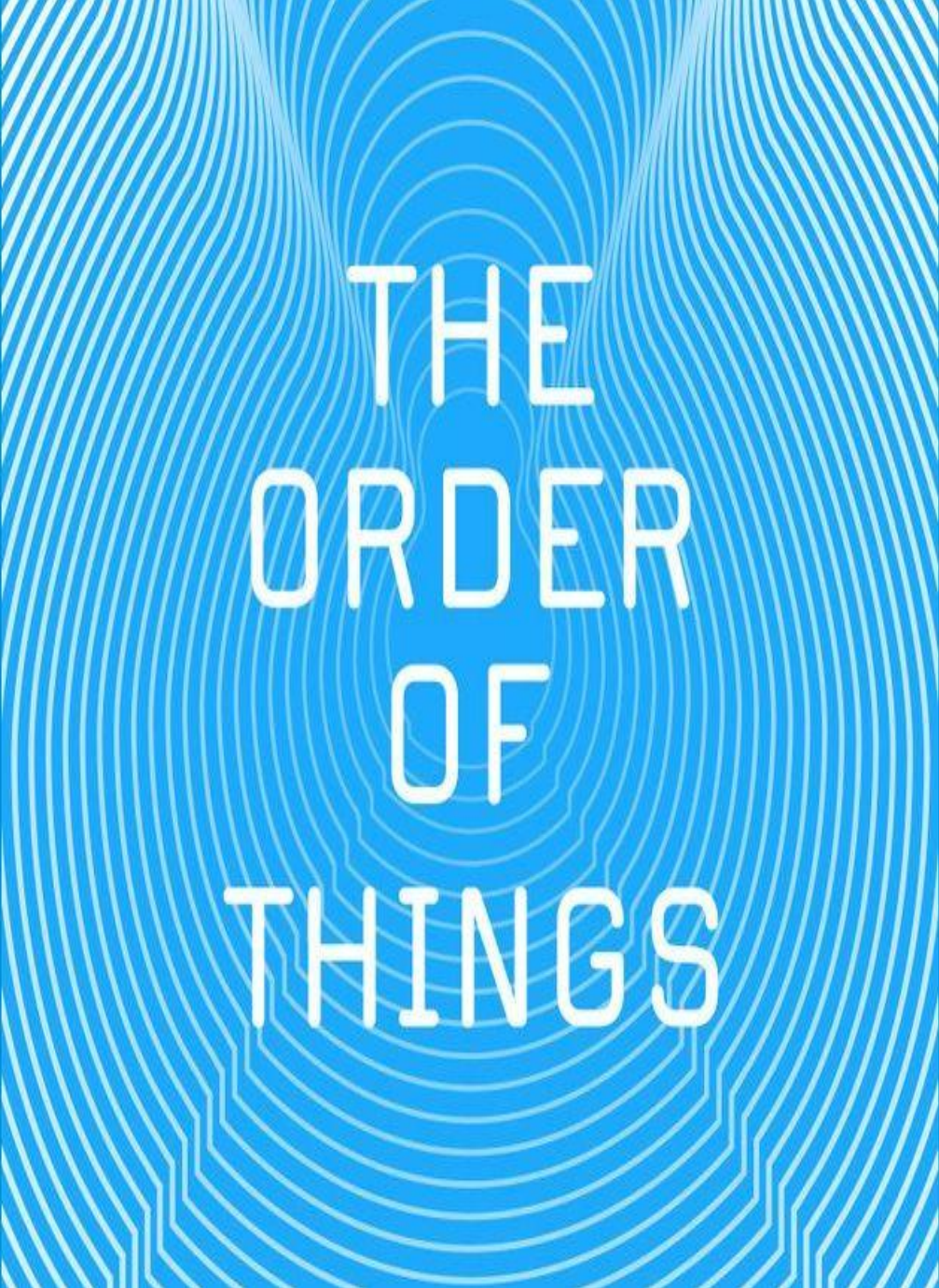
Rule 3

Rule 4

Rule 5

Rule 6

Rule 7



THE ORDER OF THINGS

Do *a*

Do *b*

Do *c*

Do *d*

Do *e*

Do *f*

Do *g*

Learning objectives

Understand how to:

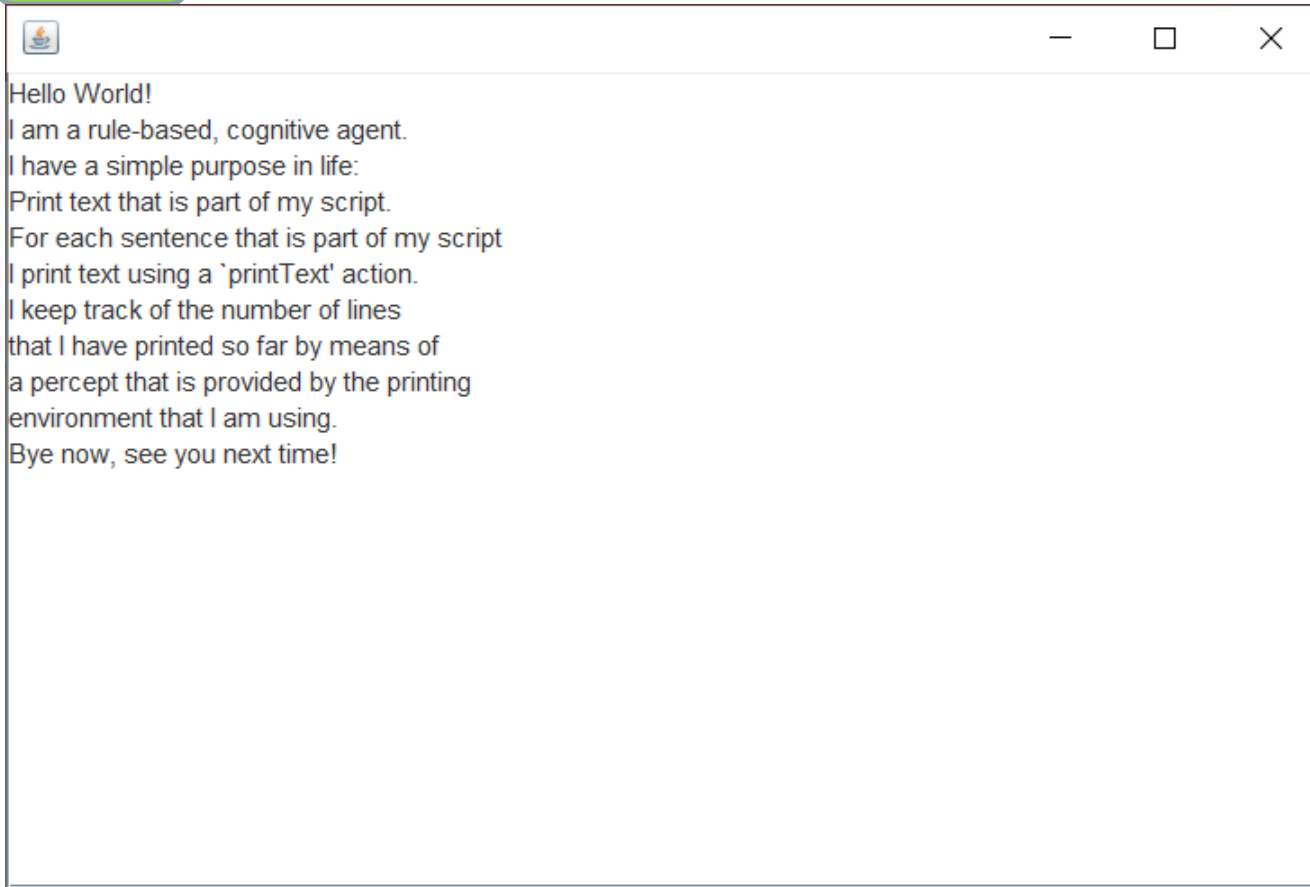
- Identify properties of environments.
- Represent and reason about environments.
- Use action rules: applicability and evaluation.
- Rule order affects agent behavior
- Set a goal: module parameters and agendas.
- Debug and refine an agent program.

Agenda

- Properties of the Hello World environment.
- Properties of the Blocks World environment.
- Representing & reasoning about the Blocks World.
- Channels for processing percepts.
- Built-in actions: an overview.
- Action rule evaluation.
- Representing a goal as module parameter or agenda.
- Solving Blocks World problems.

Recap: Hello World Environment

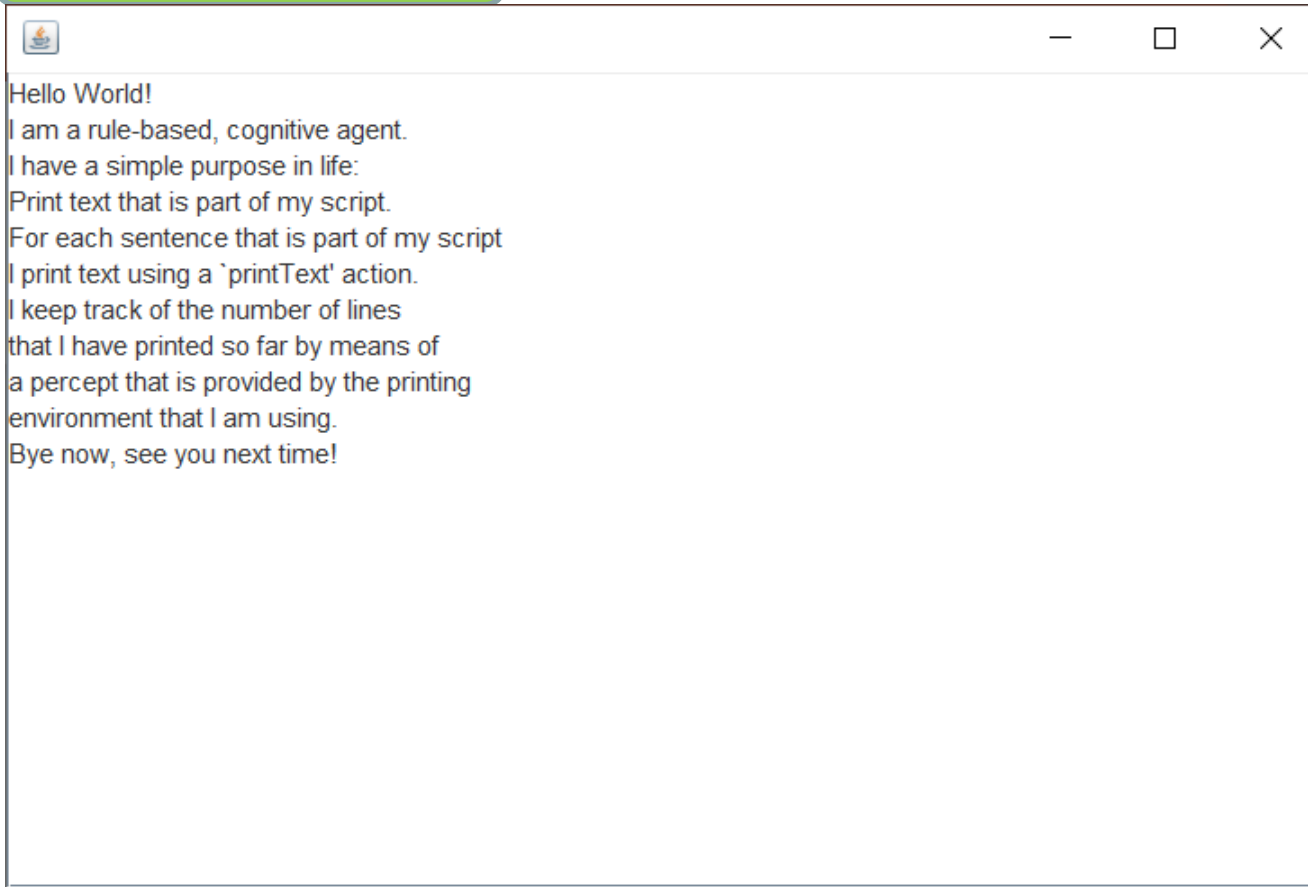
Fully or partially observable?



Agent can (not) always “see” everything **relevant for its decision making**.

Recap: Hello World Environment

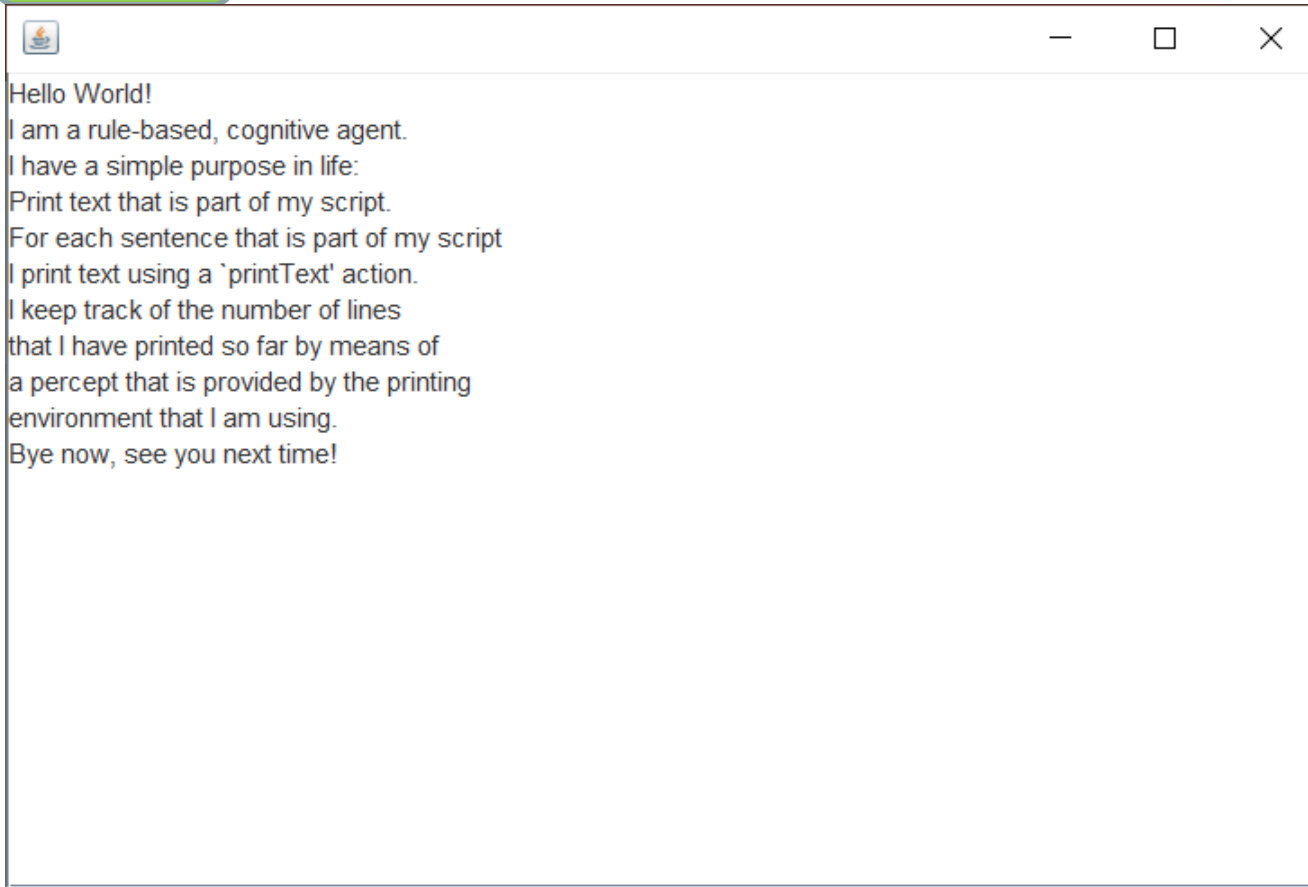
Deterministic or stochastic?



Effects of actions on the environment state are (not) completely predictable.

Recap: Hello World Environment

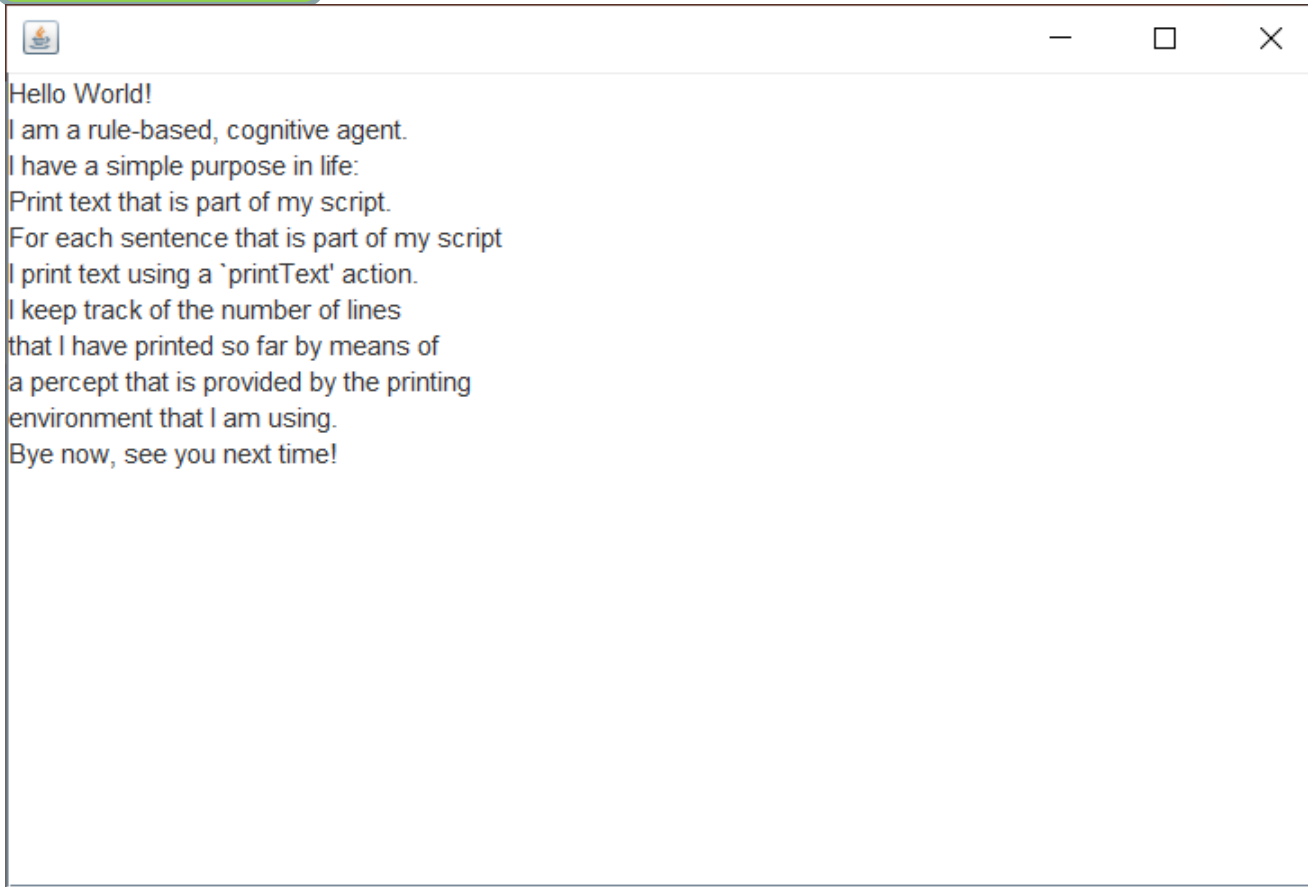
Static or dynamic?



Environment changes (not) only when an agent performs an action.

Recap: Hello World Environment

Discrete or continuous?

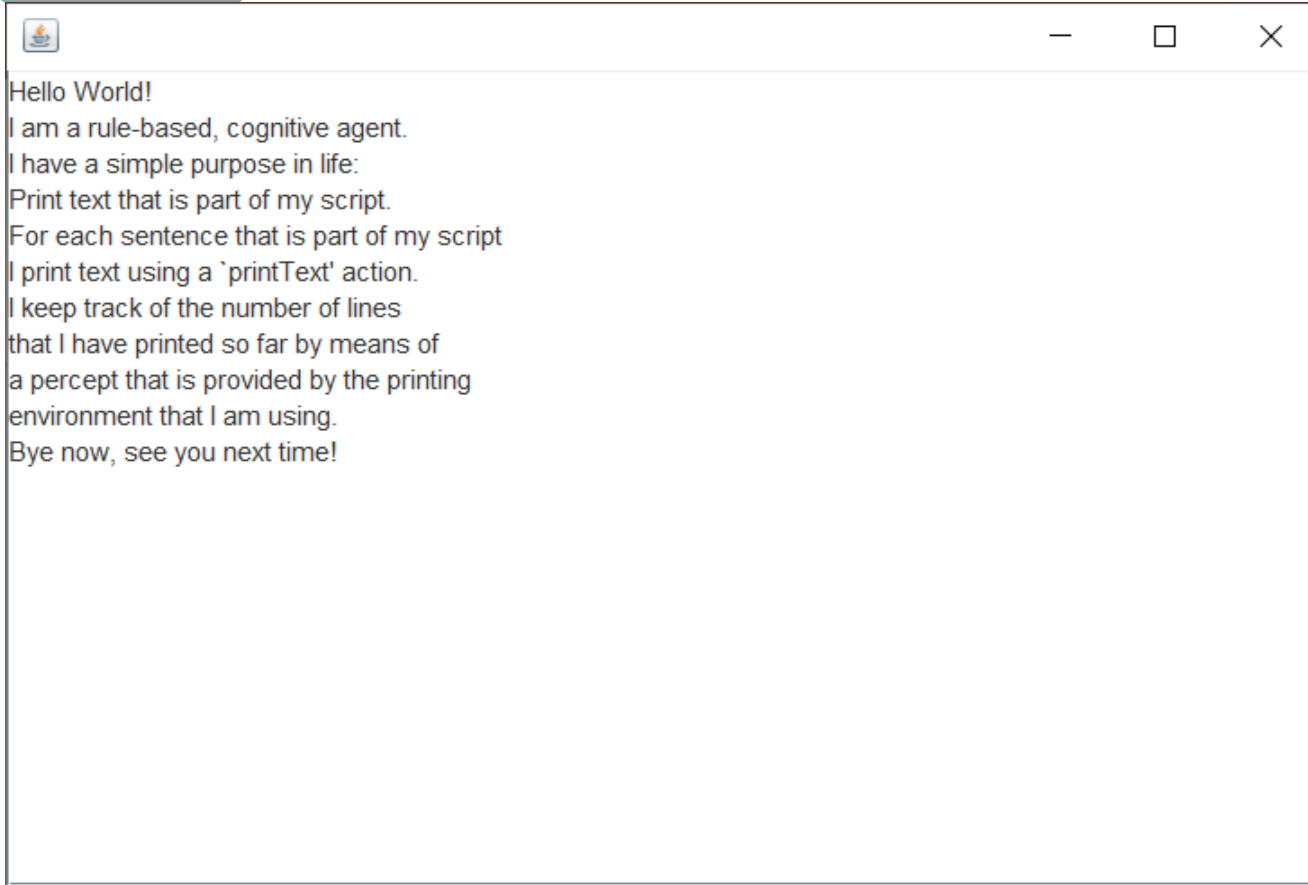


Discrete means that environment states are finite (or countable).

Environment state changes can (should) be treated as discrete (continuous).

Recap: Hello World Environment

Single or multi agent?



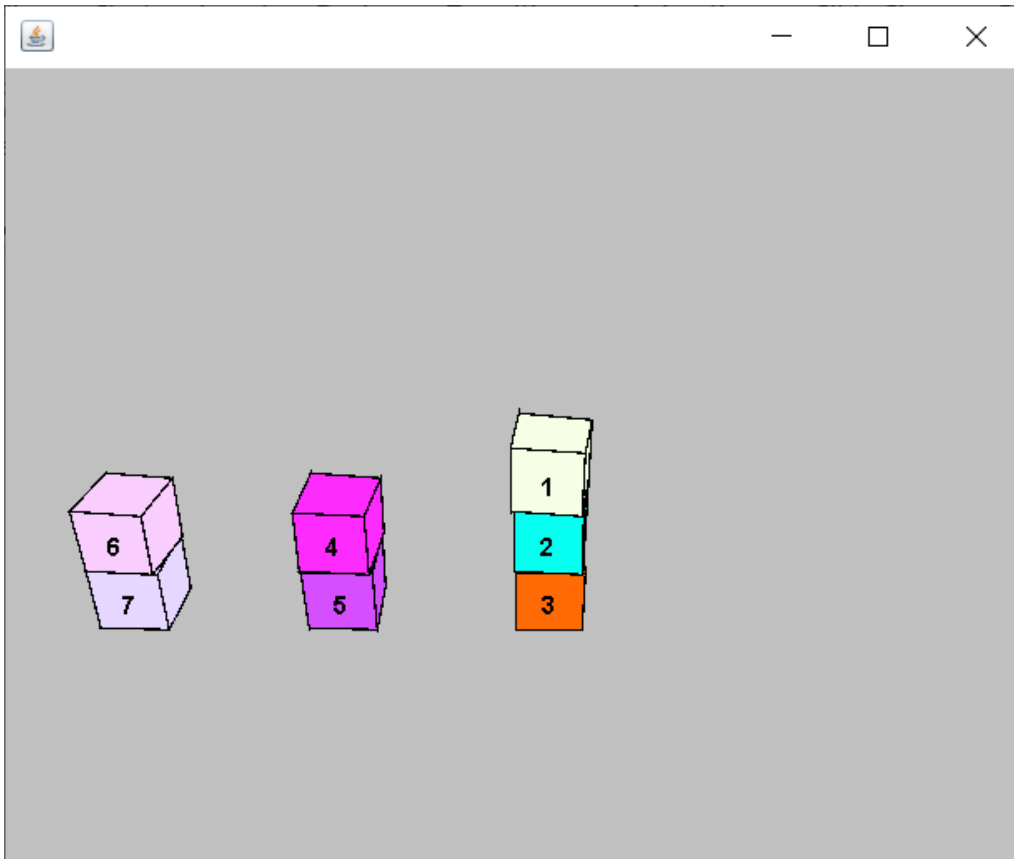
Discrete means that environment states are finite (or countable).

There is (more than) one agent performing actions changing the environment.

An Agent Program That Solves Blocks World Problems

THE BLOCKS WORLD

The Blocks World



*A classic and famous
AI planning problem.*

The Blocks World consists of **stacks** of **blocks**. These stacks are sitting on an (invisible) **table**. A block can be moved by an (invisible) **gripper** if there are no other blocks sitting on that block; such a move always is successful. There is always room on the table to move a block to. The gripper can “see” the complete configuration of blocks in the environment.

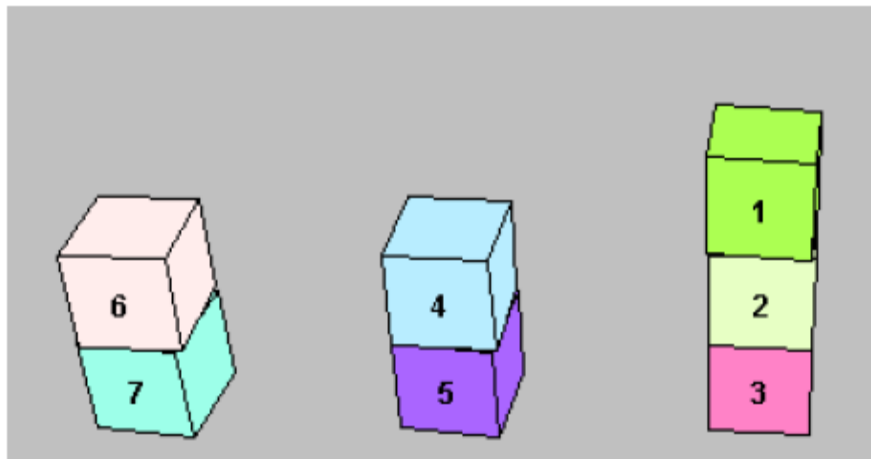
Blocks World Properties

The Blocks World consists of stacks of blocks. These stacks are sitting on an (invisible) table. A block can be moved by an (invisible) gripper if there are no other blocks sitting on that block; such a move always is successful. There is always room on the table to move a block to. The gripper can “see” the complete configuration of blocks in the environment.

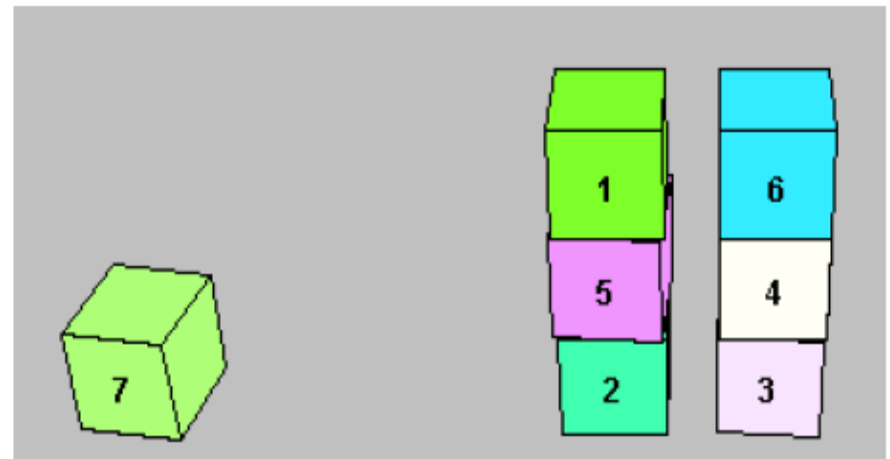
- **Fully observable:** complete configuration is accessible to gripper.
- **Deterministic:** gripper moves are completely predictable (always successful).
- **Static:** only gripper can move blocks.
- **Discrete:** environment state is a configuration of a finite number of blocks; blocks are “discrete”.
- **Single agent:** only one gripper.

Blocks World Problems

A **Blocks World Problem** is how to move blocks in a given **initial state** or configuration to achieve the **goal state**. A solution to a Blocks World problem is a sequence of actions (moves of blocks) that change the initial into the goal state.



(a) Initial State



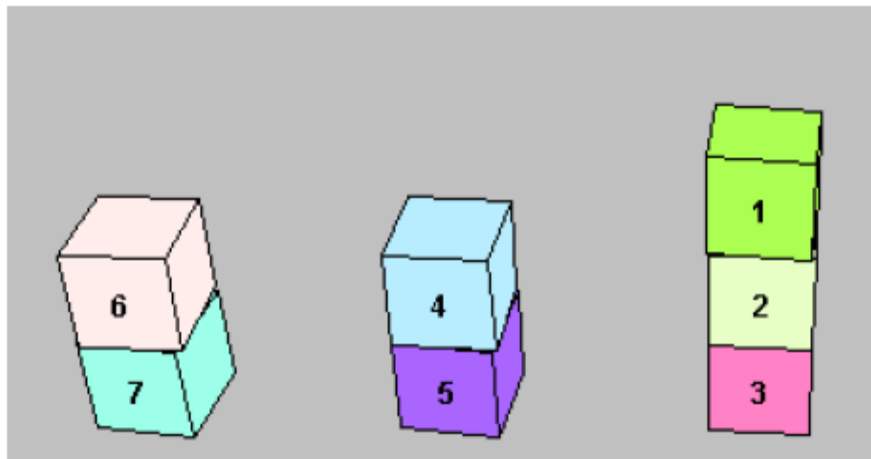
(b) Goal State

Rules of the game:

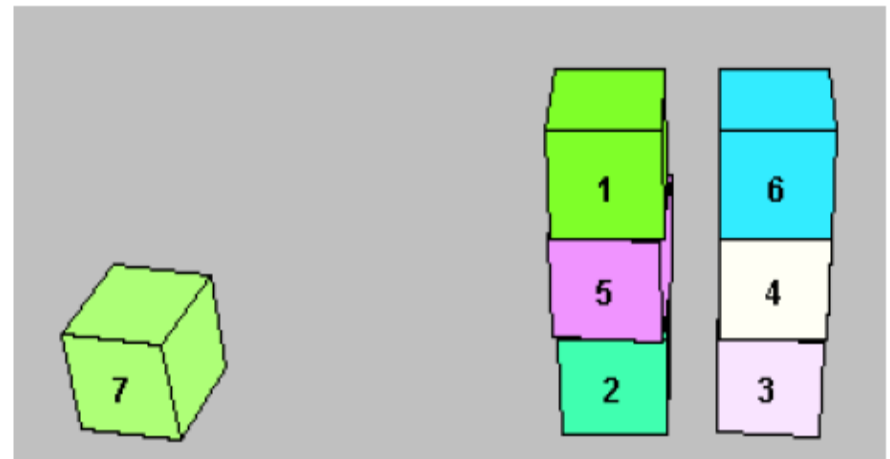
- The **positioning** of stacks on the table, whether a stack is to the left or right of another stack, is **not relevant**.
- A block is said to be **clear** if there is no other block sitting on top of it; a **block can only be moved if it is clear**.

Strategies for solving BW Problems?

A **Blocks World Problem** is how to move blocks in a given **initial state** or configuration to achieve the **goal state**. A solution to a Blocks World problem is a sequence of actions (moves of blocks) that change the initial into the goal state.



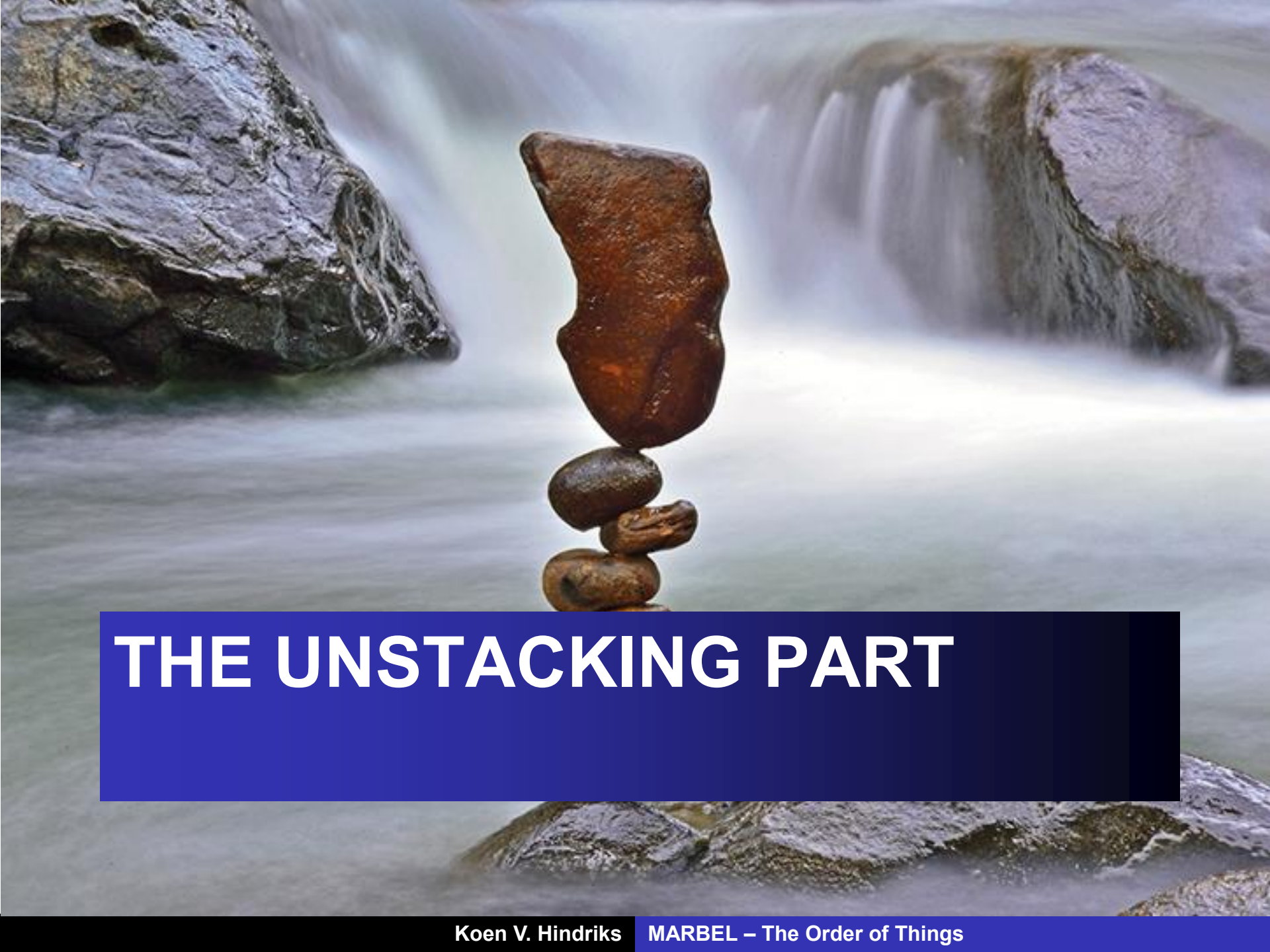
(a) Initial State



(b) Goal State

Simple strategy:

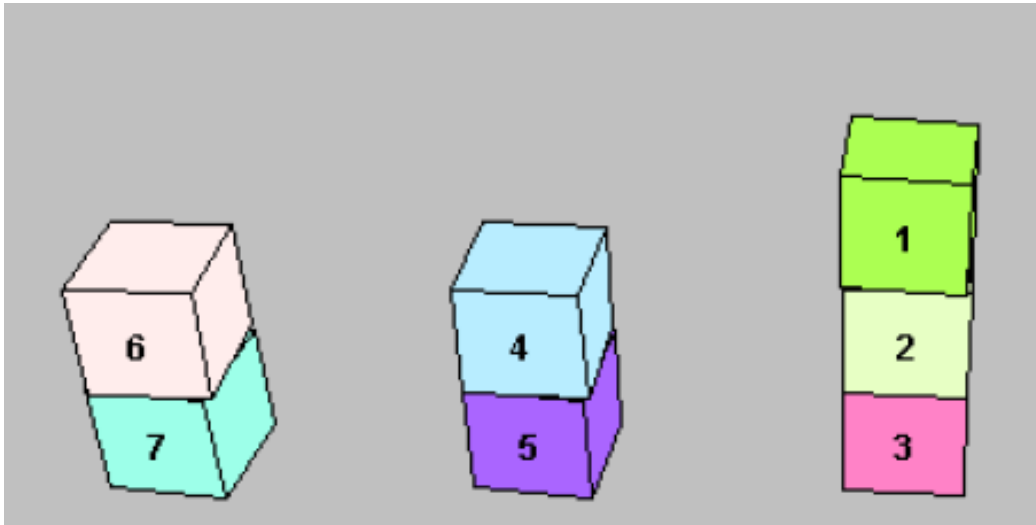
- First: **unstack**: move all blocks to the table.
- Second: **stack**: build towers to complete task.

A photograph of a rock stack in a river with a waterfall in the background. The stack consists of four smooth, dark brown stones of varying sizes, balanced precariously on top of each other. The background shows a blurred waterfall cascading over rocks, creating a sense of motion and contrast with the stillness of the rock stack. The overall scene is serene and evokes a sense of balance and stability.

THE UNSTACKING PART

Representing Blocks World States

We can **completely represent a configuration of blocks** by means of an **on/2** predicate: `on(X,Y)` means that block X sits on Y.



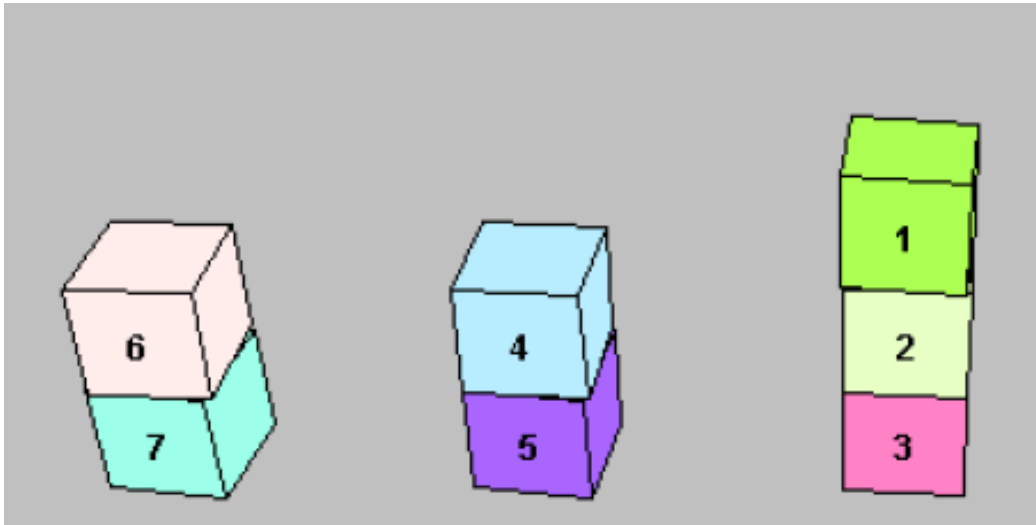
Representation:

```
on(b1,b2)
on(b2,b3)
on(b3,table)
on(b4,b5)
on(b5,table)
on(b6,b7)
on(b7,table)
```

- Blocks have **names** to be able to uniquely identify them: **b1**, **b2**, ...
- Besides block names there is an additional constant for representing the **table**.

One fact is still missing...

We can **completely represent a configuration of blocks** by means of an **on/2** predicate: `on(X,Y)` means that block X sits on Y.



Representation:

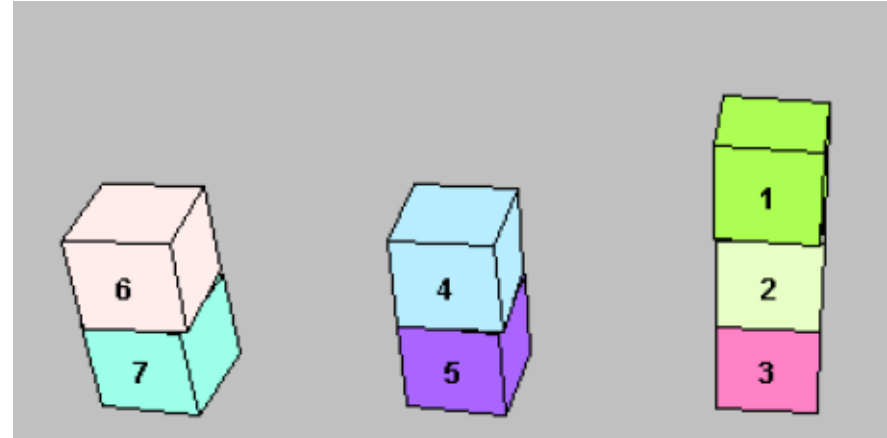
```
on(b1,b2)
on(b2,b3)
on(b3,table)
on(b4,b5)
on(b5,table)
on(b6,b7)
on(b7,table)
clear(table)
```

Which fact about the Blocks World is still missing?

Reasoning about the Blocks World

EXERCISE:

Define when X is a block.

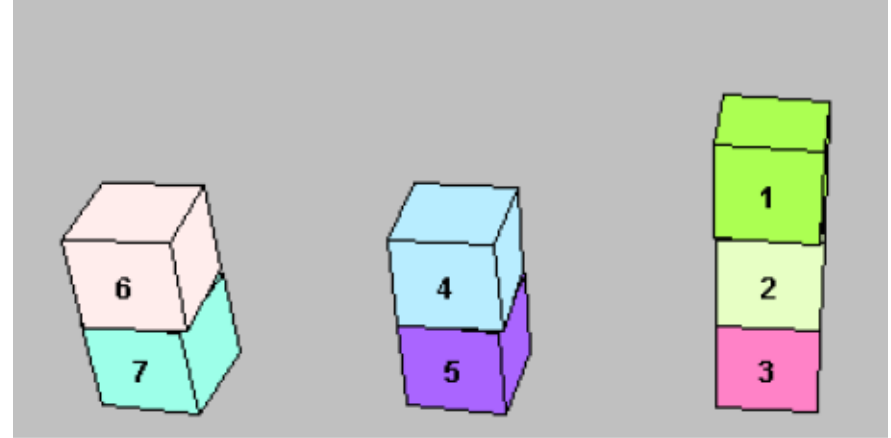


```
% only blocks can sit on something else  
block(X) :- on(X, _).
```

Reasoning about the Blocks World

EXERCISE:

Define when a block X is clear.



```
% a block is clear if nothing sits on top of it.  
clear(X) :- block(X), not( on(_,X) ).
```

Reasoning about the Blocks World

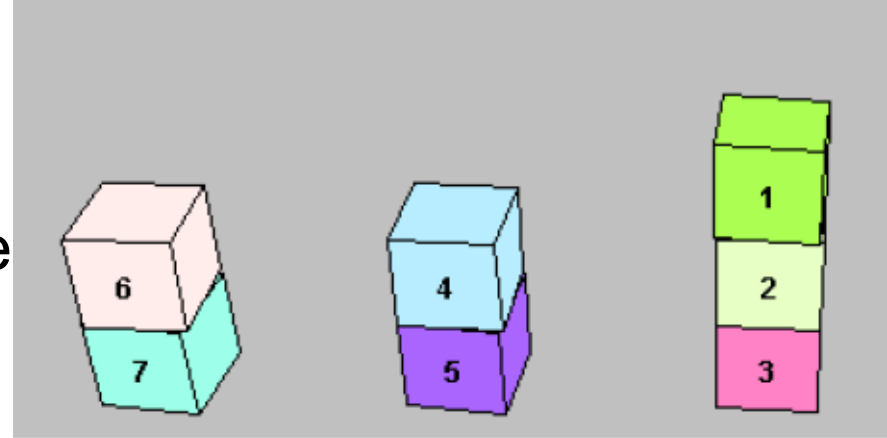
EXERCISE:

A tower is any stack of blocks that sits on the table. Towers are represented as lists. For example, [b7] and [b2, b3] are towers.

Define the tower/1 predicate.

```
tower([X]) :- on(X, table).
```

```
tower([X, Y | T]) :- on(X, Y), tower([Y | T]).
```

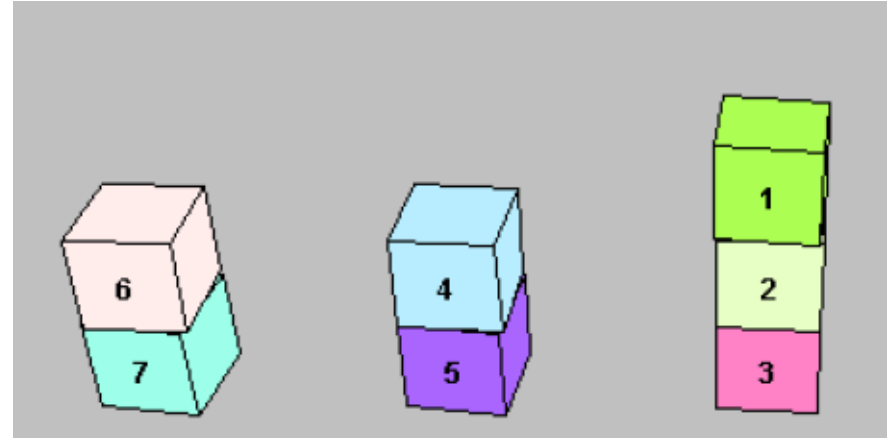


Declare predicates used but not defined

EXERCISE:

Predicates that are **used** and **not defined** must be **declared**.

Add a declaration.



```
:- dynamic on/2.
```

```
block(X) :- on(X, _).
```

```
clear(X) :- block(X), not( on(_, X) ).
```

```
clear(table).
```

```
tower([X]) :- on(X, table).
```

```
tower([X,Y|T]) :- on(X,Y), tower([Y|T]).
```

Prolog file blocksworld.pl

```
:- dynamic on/2.
```

```
block(X) :- on(X, _).
```

```
clear(X) :- block(X), not( on(_, X) ).
```

```
clear(table).
```

```
tower([X]) :- on(X, table).
```

```
tower([X,Y|T]) :- on(X,Y), tower([Y|T]).
```

Building a Blocks World mas

In mas2g file, at the top:

use "blocksworld-1.3.0.jar".

Environments can have parameters too

In mas2g file, at the top, add a 'with' clause:

use "blocksworld-1.3.0.jar"

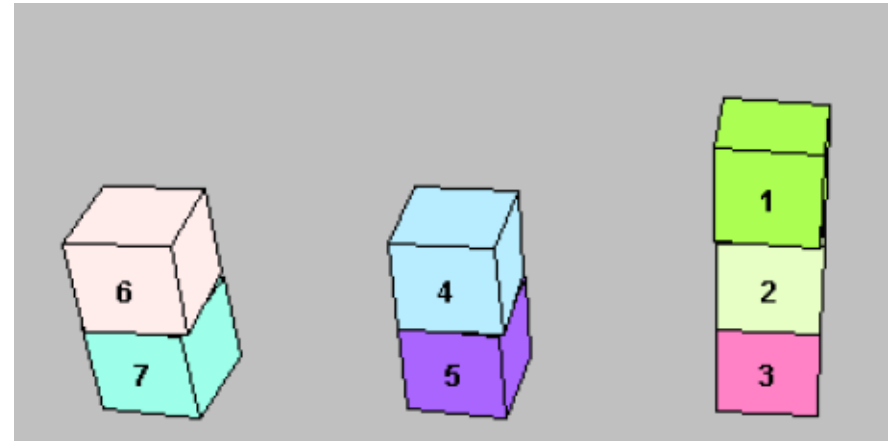
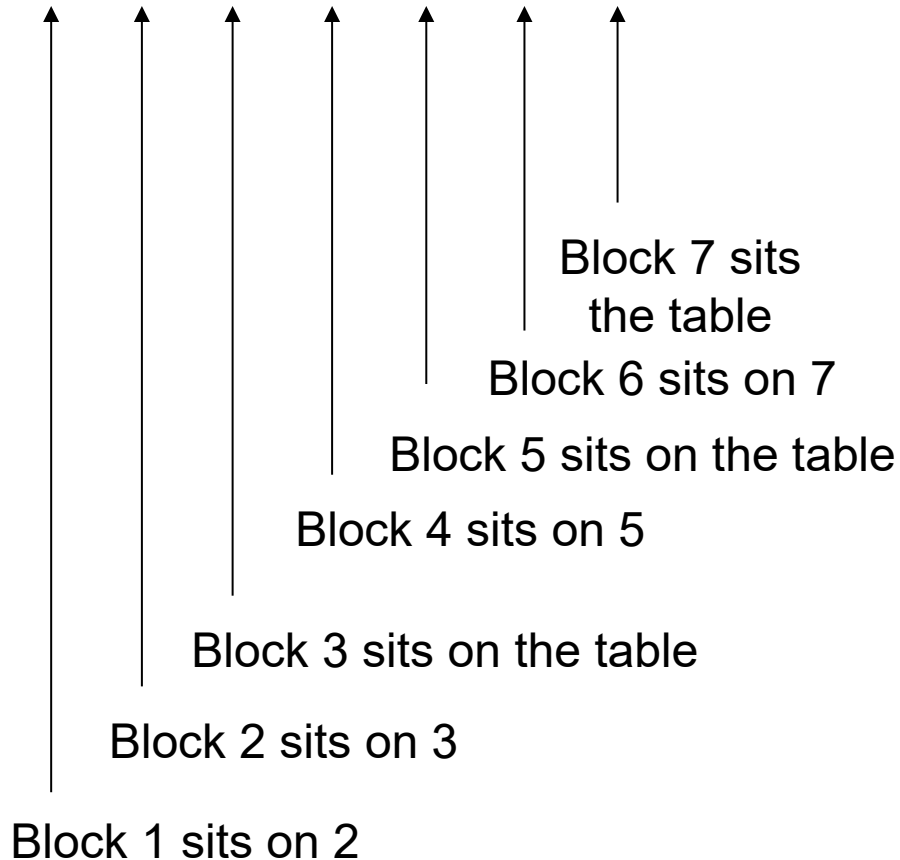
with start = [2, 3, 0, 5, 0, 7, 0].

Parameters are defined by environment. See the environment manual.

'start' parameter of BW specifies initial state of the environment.

Representing BW states as lists

[2, 3, 0, 5, 0, 7, 0]

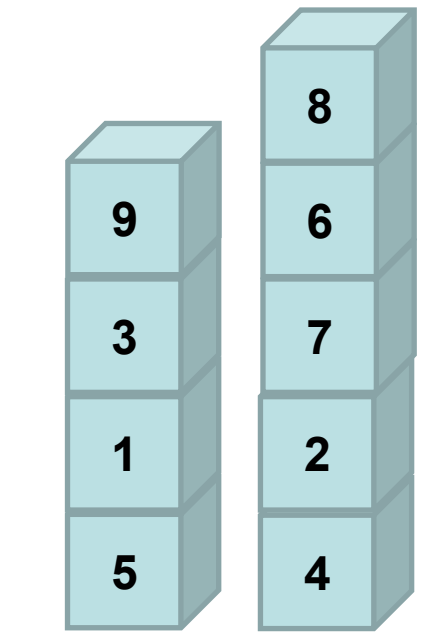
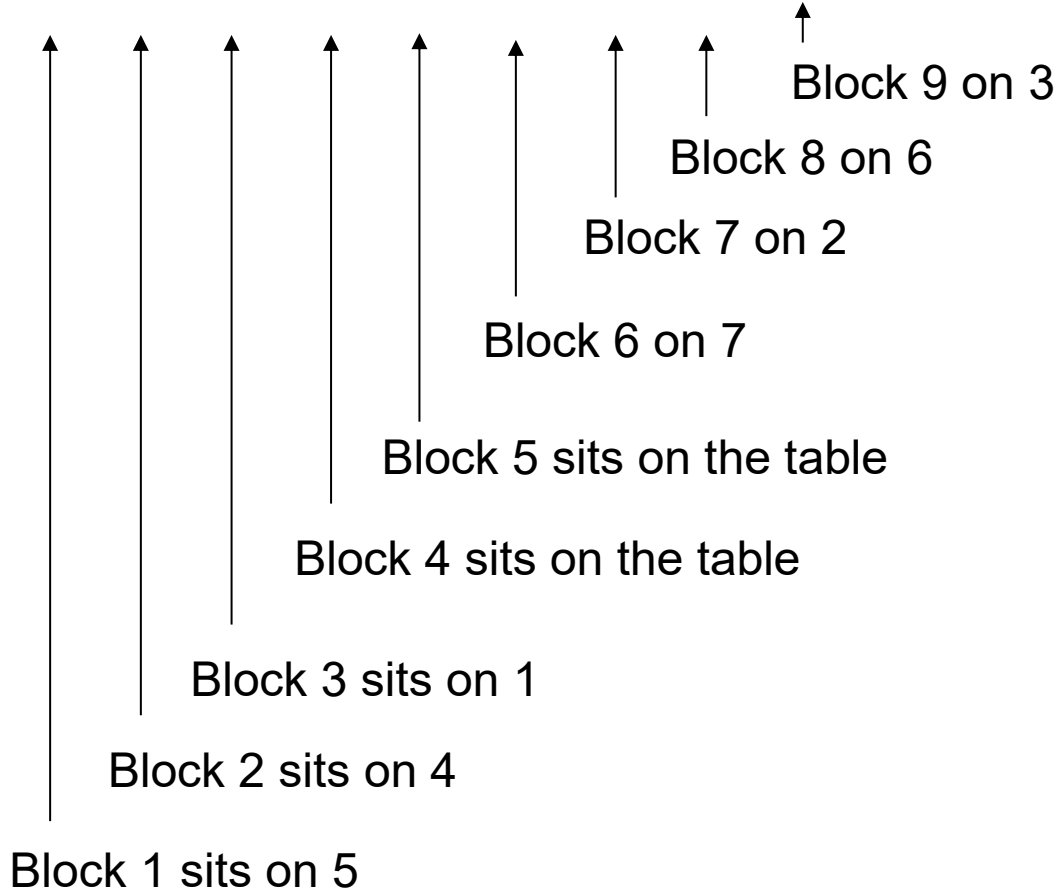


- **Index** into list for corresponding **block number** yields what the block is sitting on.
- 0 represents the **table**.

Representing BW states as lists

on: [5, 4, 1, 0, 0, 7, 2, 6, 3]

block: 1 2 3 4 5 6 7 8 9



Is this a valid BW state?

[5, 4, 1, 0, 3]

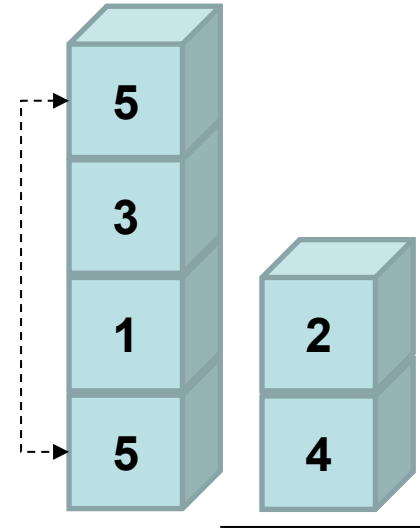
Block 5 sits on 3

Block 4 sits on the table

Block 3 sits on 1

Block 2 sits on 4

Block 1 sits on 5



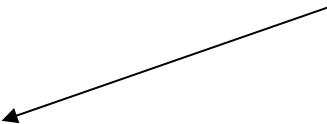
Block lists should not have cycles $5 \rightarrow 3 \rightarrow 1 \rightarrow 5!$

Building a mas: Agent definition

use "blocksworld-1.3.0.jar"

with start = [2, 3, 0, 5, 0, 7, 0].

define stackBuilderAgent {
 use stackBuilder **for** *decisions*.
}



Agent's name



Module name

Building a mas: Subscribe to channel

use "blocksworld-1.3.0.jar"

with start = [2, 3, 0, 5, 0, 7, 0].

define stackBuilderAgent {

use stackBuilder **for** *decisions*.

replace on/2.

}

Handler type

Percept predicate

Building a mas: Launch policy

use "blocksworld-1.3.0.jar"

with start = [2, 3, 0, 5, 0, 7, 0].

define stackBuilderAgent {

use stackBuilder **for** *decisions*.

replace on/2.

}

Conditional launch rule

launchpolicy {

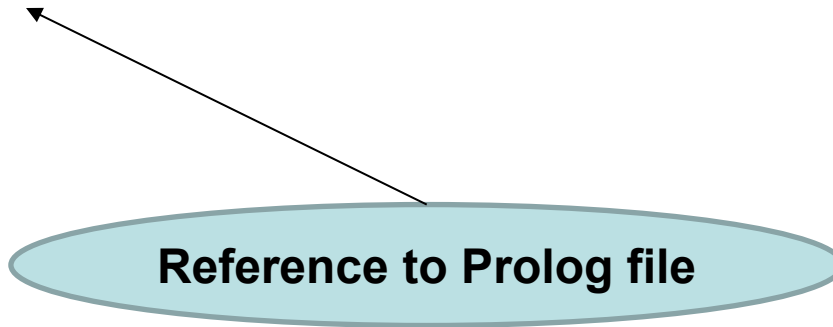
when * **launch** stackBuilderAgent.

}

Agent's name

Building a module: use clause

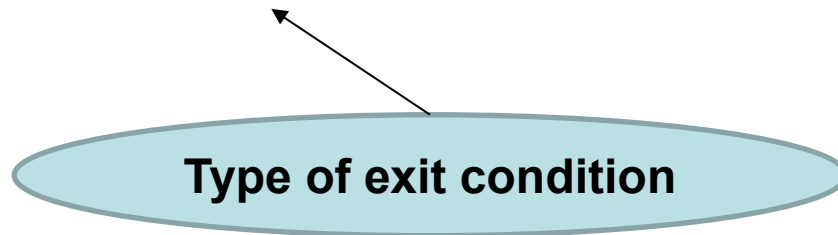
use blocksworld.



Building a module: exit condition

use blocksworld.

exit=*noaction*.



- Exit condition terminates module (and agent).
- Only use exit condition in module **for *decisions***
- **noaction**: terminate when no action is selected.

Building a module: program section

use blocksworld.

exit=*noaction*.

```
module stackBuilder {  
    if <Prolog query> then <Action>.  
}
```

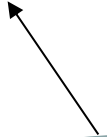
Module name (same as file!)



Action rule



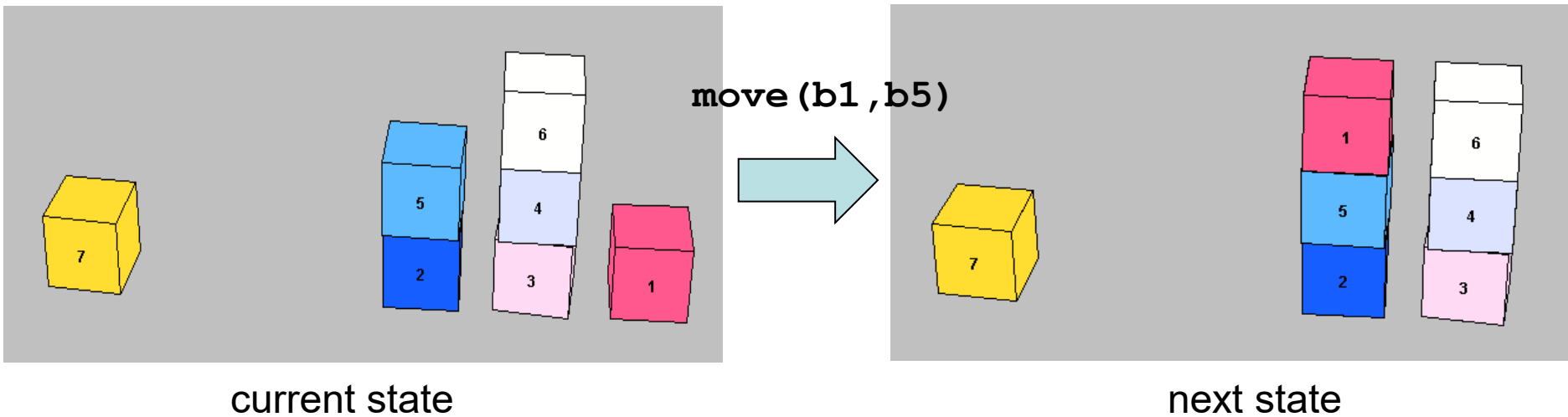
Either built-in or
environment action.



Summary: Minimal project recipe

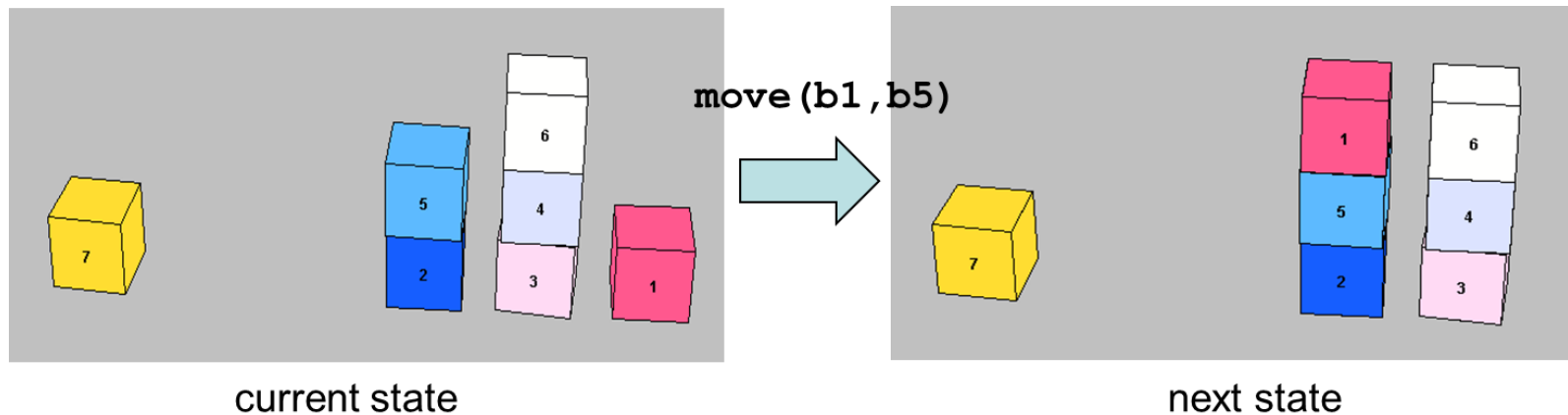
- A mas file: BlocksWorld.mas2g
 - Specifies environment connector.
 - Defines agent (stackBuilderAgent)
 - Specifies launch policy.
- A module file: stackBuilder.mod2g (**for *decisions***)
 - Specifies exit condition
 - Defines program section of module.
- A Prolog file: blocksworld.pl
 - Declares predicates that are used but not defined.
 - Defines rules (Prolog program)

Let's get moving...

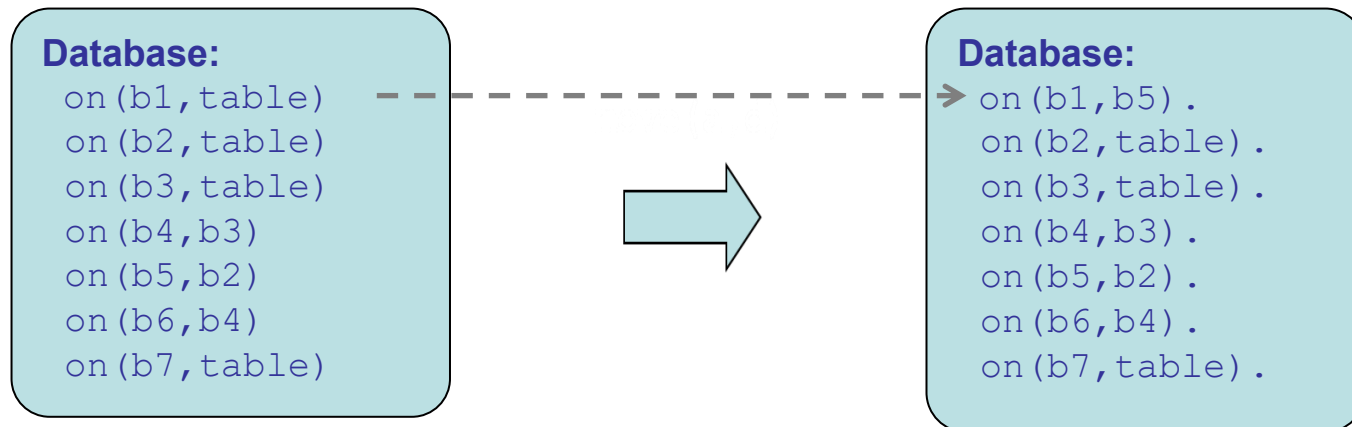


Which environment actions are available depends on the environment. Check out environment manual.

Channel processing updates database



We subscribed to channel on/2: automatically processes updates for this predicate based on percepts received from Blocks World environment.



Built-in Internal Actions

MARBEL provides a number of **built-in actions**:

- `insert(<conjunction>)`

example: `insert(on(X,Y), not(on(X,Z)))`

meaning: **add** `on(X,Y)` and **delete** `on(X,Z)`

- `delete(<conjunction>)`

example: `delete(on(X,Y), not(on(X,Z)))`

meaning: **delete** `on(X,Y)` and **add** `on(X,Z)`

Note

All actions need to be fully instantiated when they are executed.

Built-in Action Insert

Example 1:

Database:

```
on(a, table)
on(b, table)
on(c, table)
```

insert(on(a,b),
not(on(a,table)))



Database:

```
on(a,b)
on(b, table)
on(c, table)
```

Example 2:

Database:

```
on(a, table)
on(b, table)
on(c, table)
```

insert(on(d, table))



Database:

```
on(a, table)
on(b, table)
on(c, table)
on(d, table)
```

Preferred use is Example 2: Use **insert** only for inserting.

Built-in Action Delete

Example 1:

Database:

```
on(a,b)
on(b,table)
on(c,table)
```

delete (on(a,b),
 not (on(a,table)))



Database:

```
on(a,table)
on(b,table)
on(c,table)
```

Example 2:

Database:

```
on(a,table)
on(b,table)
on(c,table)
```

delete (on(a,table))



Database:

```
on(b,table)
on(c,table)
```

Preferred use: Example 2.

Further Built-in Actions

- **print(<Term>)**
prints term term to the standard console.
- **log(<Parameter>)**
writes logging information to file
(useful for debugging or storing results)

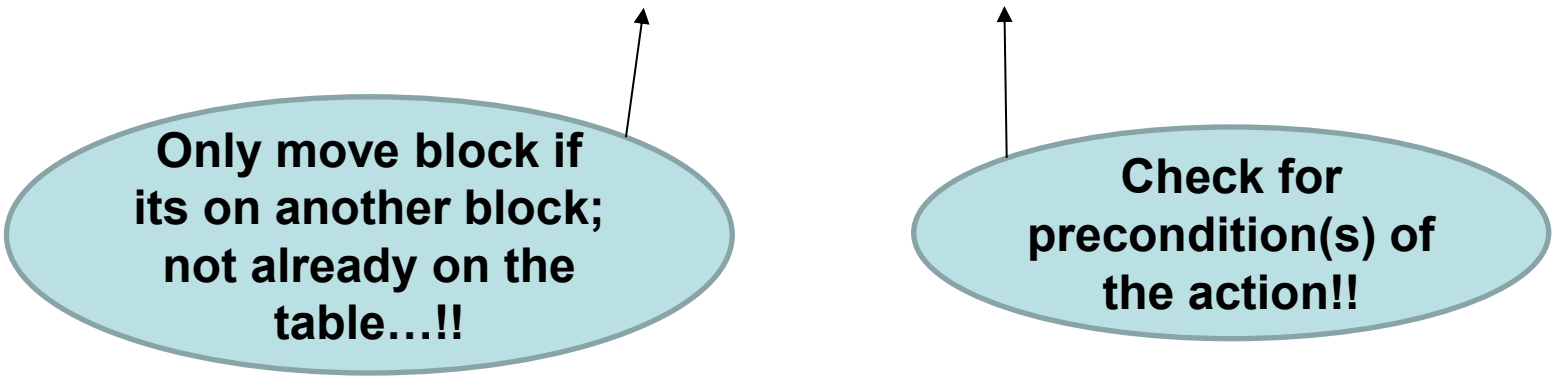
Let's get moving: Unstacking

Action rule:

if ... then move(X, table).

What query should we use to fill in ...?

on(X, Y), block(Y), clear(X)



**Only move block if
its on another block;
not already on the
table...!!**

**Check for
precondition(s) of
the action!!**

Building a module: program section

use blocksworld.

exit=*noaction*.

```
module stackBuilder {  
    if on(X, Y), block(Y), clear(X)  
        then move(X, table).  
}
```

Action Rules

Generic form:

```
if <query> then <action> (+ <action2> ...) .
```

Example:

```
if on(X, Y), not(Y=table) then move(X,table) .
```

Evaluating a rule:

1. Evaluate query: check if query succeeds on database; **if so**,
2. Select binding of variables for which query succeeds;
3. Instantiate action;
4. Perform instantiated action.

Example: Action Rule Evaluation

```
if on(X, Y), not(Y=table) then move(X,table).
```

Database:

```
block(X) :- on(X, _).  
clear(X) :- block(X), not( on(_,X) ).  
clear(table).  
tower([X]) :- on(X, table).  
tower([X,Y|T]) :- on(X,Y), tower([Y|T]).  
  
on(b1,b2).  
on(b2,b3).  
on(b3,table).  
on(b4,b5).  
on(b5,table).  
on(b6,b7).  
on(b7,table).
```

1. Evaluate query: [X/b1,Y/b2], [X/b2,Y/b3], [X/b4,...], ...
2. By default, left-most binding is selected: [X/b1,Y/b2].
3. Instantiate action: move(b1,table).
4. Perform instantiated action move(b1,table).

Applicable Action Rules

An action rule

```
if <query> then <action1> + <action2> ... .
```

is said to be **applicable** if the **query succeeds**.

Is this rule applicable?

EXERCISE:

```
if on(b2,b3), not( on(b1,b3) ) then move(X, table).
```

Database:

```
block(X) :- on(X, _).  
clear(X) :- block(X), not( on(_,X) ).  
clear(table).  
tower([X]) :- on(X, table).  
tower([X,Y|T]) :- on(X,Y), tower([Y|T]).  
  
on(b1,b2).  
on(b2,b3).  
on(b3,table).  
on(b4,b5).  
on(b5,table).  
on(b6,b7).  
on(b7,table).
```

Yes: on(b2, b3) in database; on(b1, b3) not.

Is this rule applicable?

EXERCISE:

```
if on(X,table), on(Y,X), not( clear(Y) ) then move(X, table).
```

Database:

```
block(X) :- on(X, _).  
clear(X) :- block(X), not( on(_,X) ).  
clear(table).  
tower([X]) :- on(X, table).  
tower([X,Y|T]) :- on(X,Y), tower([Y|T]).  
  
on(b1,b2).  
on(b2,b3).  
on(b3,table).  
on(b4,b5).  
on(b5,table).  
on(b6,b7).  
on(b7,table).
```

Yes: [X/b3, Y/b2].

Is this rule applicable?

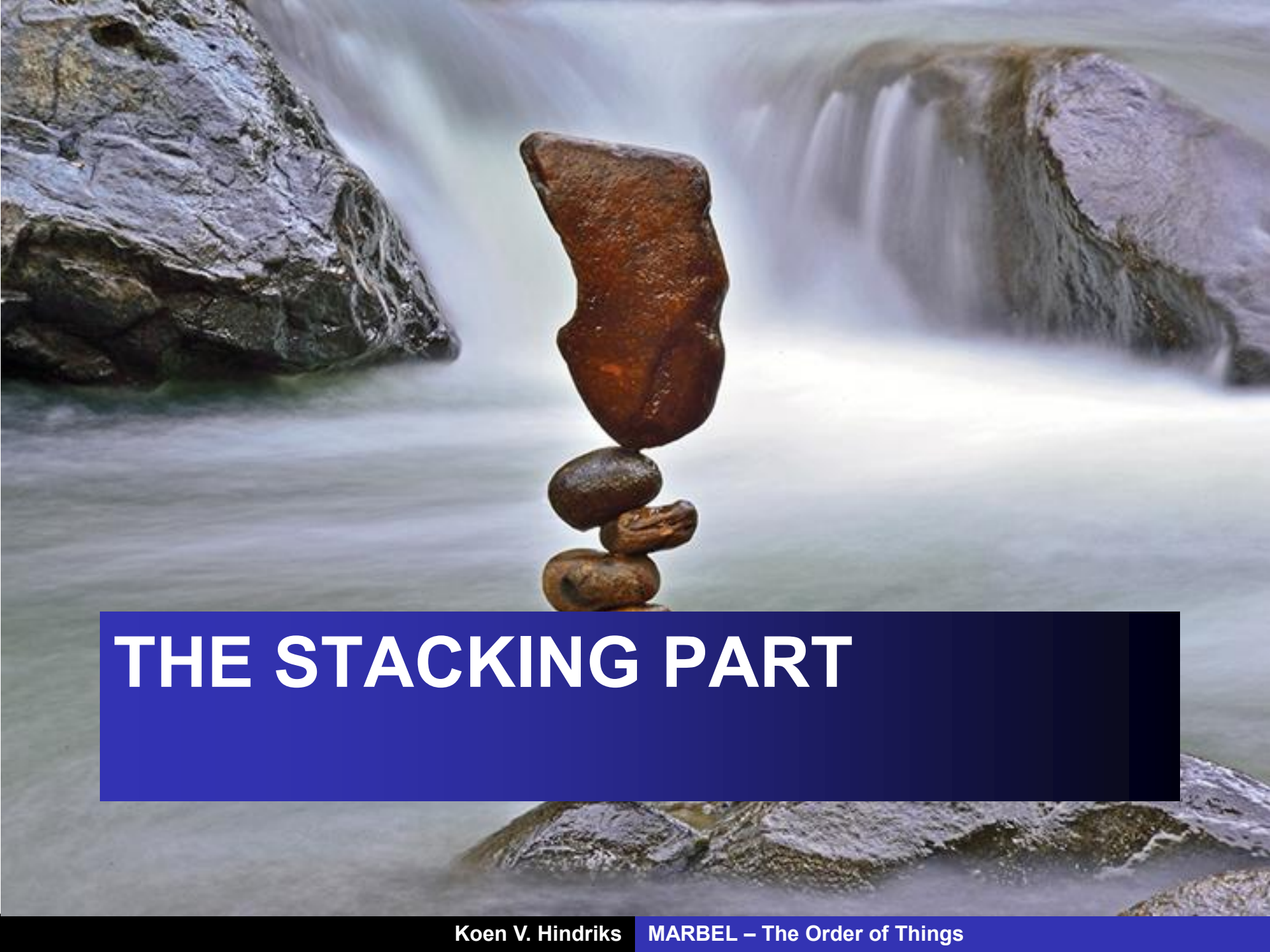
EXERCISE:

```
if tower([X,b2,b4]) then move(X, table).
```

Database:

<code>block(X) :- on(X, _).</code>	<code>on(b1,b2) .</code>
<code>clear(X) :- block(X), not(on(_,X)).</code>	<code>on(b2,b3) .</code>
<code>clear(table) .</code>	<code>on(b3,table) .</code>
<code>tower([X]) :- on(X, table) .</code>	<code>on(b4,b5) .</code>
<code>tower([X,Y T]) :- on(X,Y), tower([Y T]) .</code>	<code>on(b5,table) .</code>
	<code>on(b6,b7) .</code>
	<code>on(b7,table) .</code>

No: b2 sits on b3, not b4.

A long-exposure photograph of a waterfall, creating a soft, blurred effect for the water. In the foreground, a stack of four smooth, dark brown stones is balanced vertically. The background shows large, dark rocks on either side of the waterfall. A dark blue rectangular box is overlaid on the lower half of the image, containing the title text.

THE STACKING PART

Setting a goal

For the unstacking part, information about the initial state came from the environment. **For the stacking part we need to tell the agent what the goal state is.**

Two questions we need to answer:

- How to provide the agent with this info?
- How to represent the goal state?

How to set a goal for an agent?

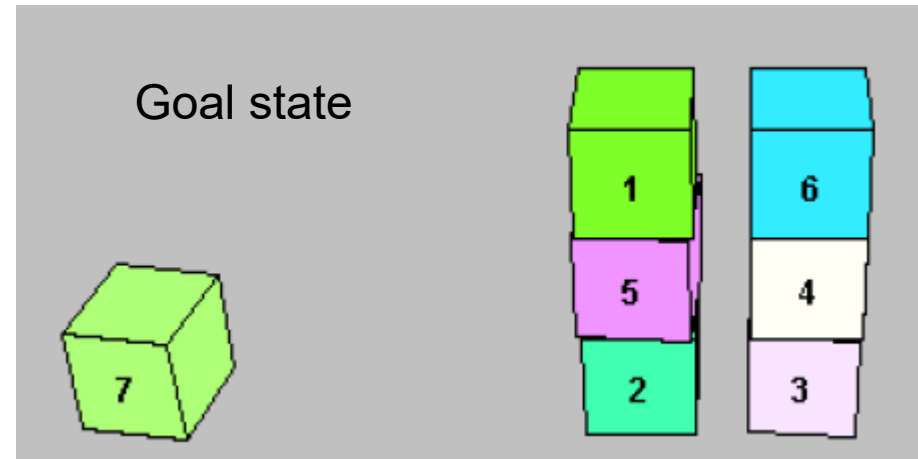
One way to do it: **use a module parameter to pass the goal state as an argument.**

In `stackBuilder.mod2g` file:

```
module stackBuilder(Goal) {  
    ...  
}
```

How to represent the goal?

Module parameters need to be instantiated with a term:
use a list representation of the goal state.



In BlocksWorld.mas2g file:

```
define stackBuilderAgent {  
    use stackBuilder([b5, table, table, b3, b2, b4, table])  
    for decisions.
```

```
    ...
```

```
}
```

Which block should be on another?

EXERCISE:

Suppose: Goal=[b5, table, table, b3, b2, b4, table]

Write a predicate that succeeds for on(Goal, b1, b5), etc.

Define:

```
on(Goal, X, Y) :-  
    nth1(Index, Goal, Y), atom_concat(b, Index, X) .
```

Hint:

- **use:** nth1(?Index, ?List, ?Elem);
example: nth1(2, [a, b, c], b) **succeeds.**
- **use:** atom_concat(?Atom1, ?Atom2, ?Atom3);
example: atom_concat(b, 1, b1) **succeeds.**

A rule for getting what we want

Action rule for achieving (part of the) goal:

if on(Goal, X, Y), not(on(X, Y)),

clear(Y), clear(X)

then move(X, Y).

Avoid useless moves

Check that we do
not already have
what we want.

Check for
preconditions of
the action!

Make sure the move can be made

Combining rules... The Order of Things

1

```
module stackBuilder {
```

```
    if on(X, Y), block(Y), clear(X) then move(X, table).
```

```
    if on(Goal, X, Y), not(on(X, Y)), clear(Y), clear(X)  
        then move(X, Y).
```

```
}
```

2

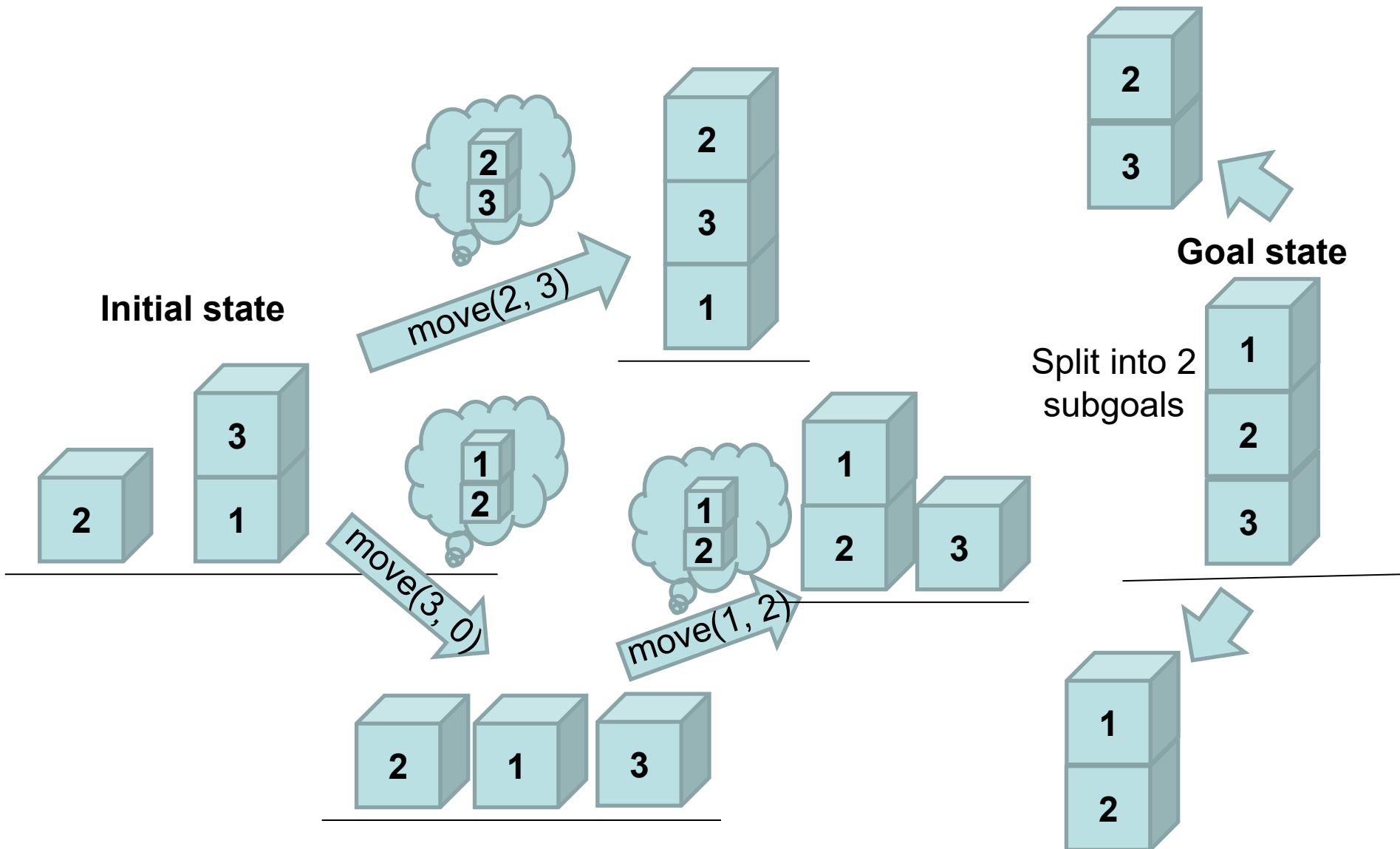
```
module stackBuilder {
```

```
    if on(Goal, X, Y), not(on(X, Y)), clear(Y), clear(X)  
        then move(X, Y).
```

```
    if on(X, Y), block(Y), clear(X) then move(X, table).
```

```
}
```

The Sussman Anomaly



How to use goal parameter?

How to compute which tower(s) agent needs to construct from goal parameter?

EXERCISE:

Define `tower(Goal, T)`. **Use** `on(Goal, X, Y)`.

```
tower(Goal, [X]) :-  
    on(Goal, X, table).  
  
tower(Goal, [X, Y|T]) :-  
    on(Goal, X, Y), tower(Goal, [Y|T]).
```


Making constructive moves

Using tower/3 (what we want) and tower/2 (what is the case now), we can define a **constructive move**:

move(X, Y) **is constructive** if it moves block X into position: on top of the tower that we need it to be on.

The following query implements this concept:

```
tower(Goal, [X, Y | T]), tower([Y | T]),  
clear(X), clear(Y)
```

Use this query instead in our rule for moving a block onto another block.

Only move misplaced blocks to the table

Almost there... our agent still keeps moving blocks to the table after achieving the goal...

A block **is misplaced** if it sits on top of the wrong block.

The following query implements this concept:

```
tower(Goal, [X|T]), not( tower([X|T]) ),  
clear(X), not( on(X, table) )
```

Use this query instead in our rule for moving a block to the table.

Strategy for Solving BW Problem

Summarizing informally the **strategy** of our agent for solving Blocks World problems:

1. If a **constructive** move can be made, make it.
2. If a block is **misplaced**, move it to the table.

Note that the order makes a difference:

- First 1 then 2 $\rightarrow$ agent needs 7 cycles.
- First 2 then 1 $\rightarrow$ agent needs 9 cycles.

The Order of Things

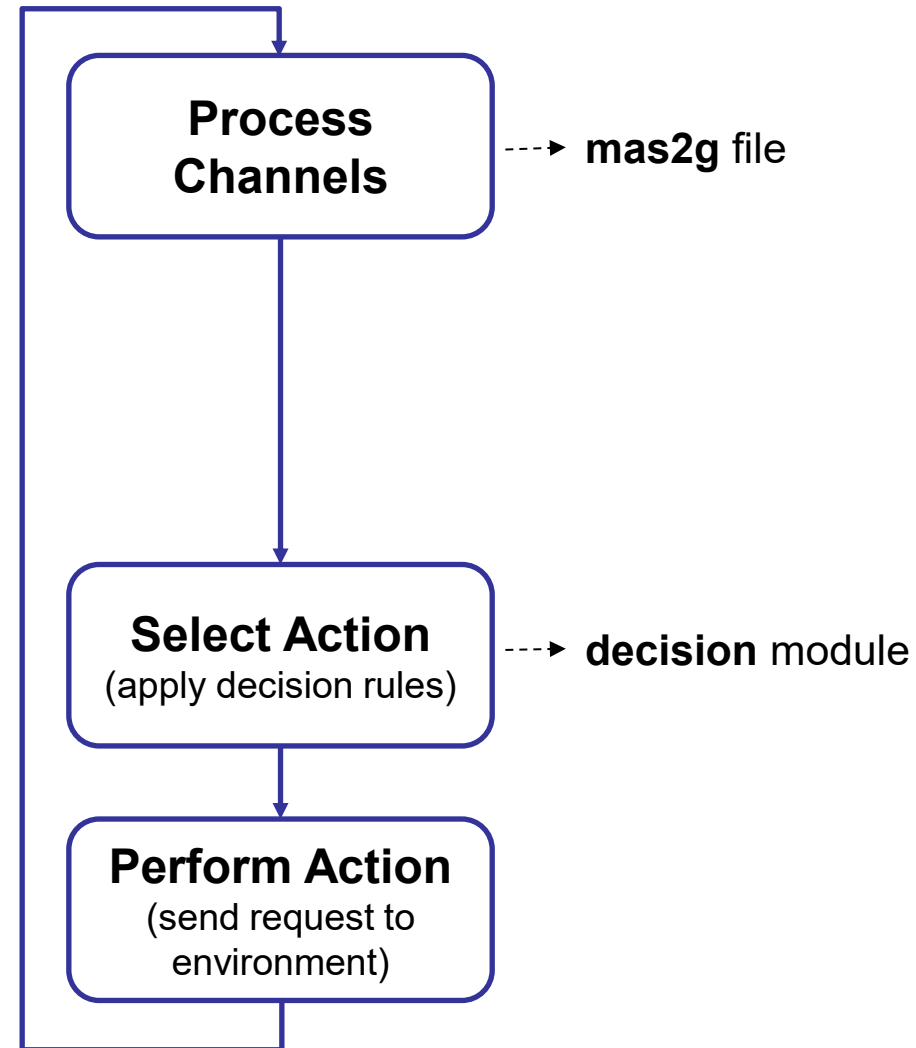
```
module stackBuilder(Goal) {  
    % make a constructive move whenever possible  
    if tower(Goal, [X, Y | T]), tower([Y | T]), clear(X), clear(Y)  
        then move(X, Y).  
    % move a misplaced block to the table  
    if tower(Goal, [X | T]), not( tower([X | T]) ), clear(X), not( on(X, table) )  
        then move(X, table).  
}
```

How is a module (for *decisions*) executed?

1. Starting from top to bottom, find first rule that is applicable;
2. Apply that rule;
3. Exit the module;
4. Continue with the agent's execution cycle.

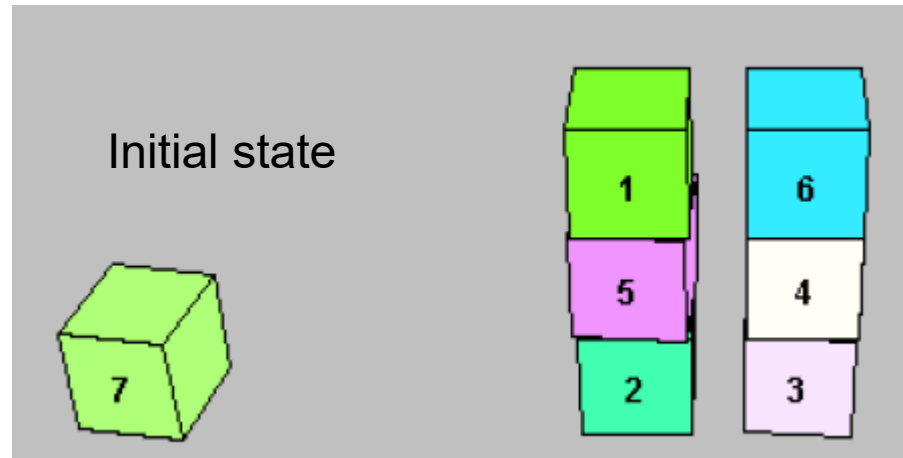
Agent Cycle: Blocks World Example

- ~~Start: **init** module:~~
 - ~~— Initialize database state~~
- Process: **channels**
 - Process percepts
 - ~~— Process messages~~
- ~~Process: **update** module~~
 - ~~— Update database state~~
- Select: **decision** module
 - Action selection (strategy)
- Perform: **action**:
 - Request env to execute action



Blocks that are not part of the goal

Does our agent solve all Blocks World configurations?



Suppose we have $\text{Goal} = [\text{table}, b1, b2]$

Are blocks $b4$, $b5$, $b6$ misplaced?

HOMEWORK ASSIGNMENT:

Think about a solution. *Hint:* define **above** (x , y)



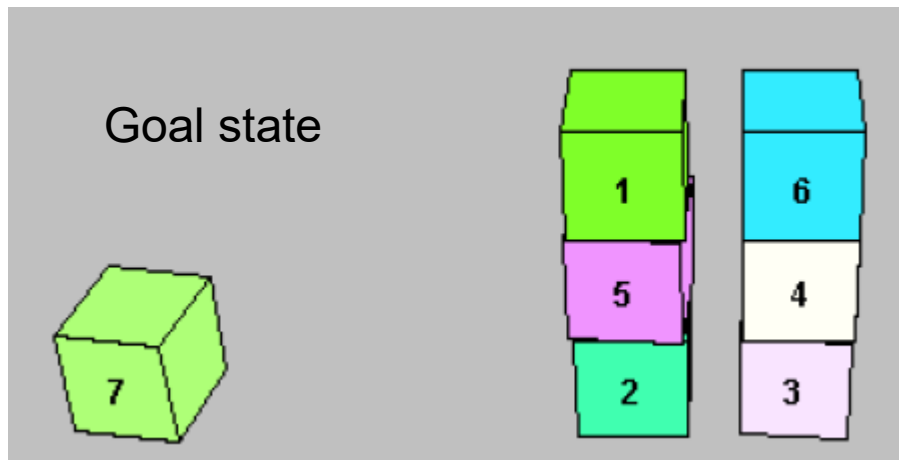
Using an agenda instead of a module parameter for setting a goal.

ALTERNATIVE GOAL SETTING

Declaring an agenda

In blocksworld.pl:

`:- dynamic agenda/1, on/2.`



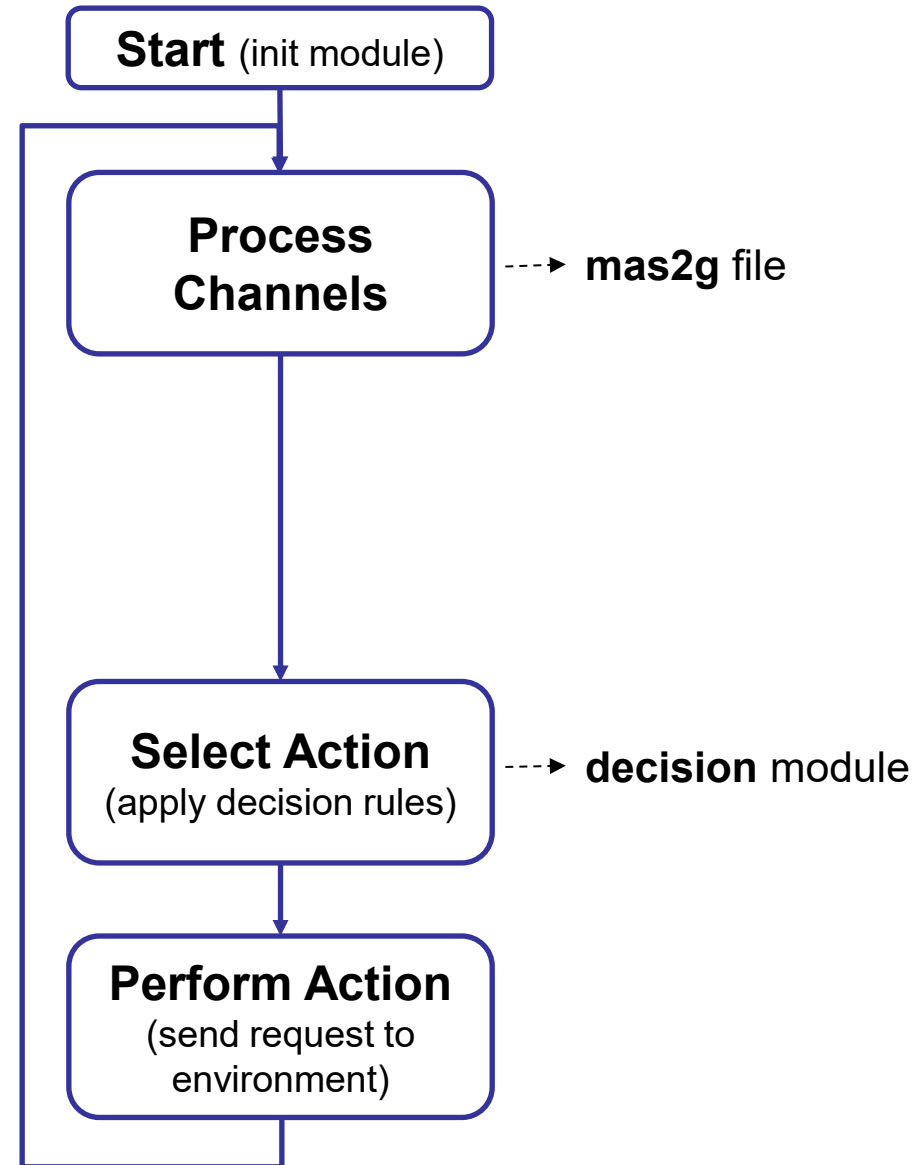
Basic idea:

insert agenda into database when agent is launched:

`agenda([b5,table,table,b3,b2,b4,table])`

Init module for setting a goal

- Start: **init** module:
 - Initialize database state
- Process: **channels**
 - Process percepts
 - ~~Process messages~~
- ~~Process: **update** module~~
 - ~~Update database state~~
- Select: **decision** module
 - Action selection (strategy)
- Perform: **action**:
 - Request env to execute action



Add init module to agent definition

use "blocksworld-1.3.0.jar"

with start = [2, 3, 0, 5, 0, 7, 0].

define stackBuilderAgent {

use stackBuilderinit for *init*.

use stackBuilder for *decisions*.

replace on/2.

}

...

**Remove module
parameter.**

Initializing an agent's database

Create `stackBuilderInit.mod2g`:

use blocksworld.

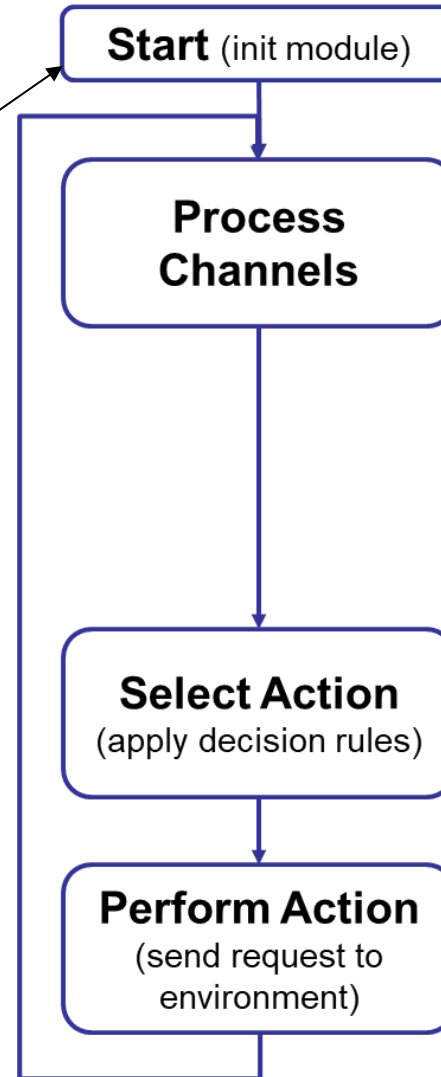
```
module stackBuilderInit {  
  if true  
    then insert(agenda([b5,table,table,b3,b2,b4,table])).  
}
```

Init modules do not have exit conditions

Why not?

Inspect agent cycle:

**Init module is
executed only
once.**



Using agenda instead of Goal parameter

We need to slightly change our Prolog definitions for reasoning about towers:

```
agenda-tower([X]) :-  
    agenda(Goal, on(Goal, X, table)).  
agenda-tower([X,Y|T]) :-  
    agenda(Goal, on(Goal, X, Y)),  
    agenda-tower([Y|T]).
```

Replace `tower/2` **everywhere with** `agenda-tower/1`.

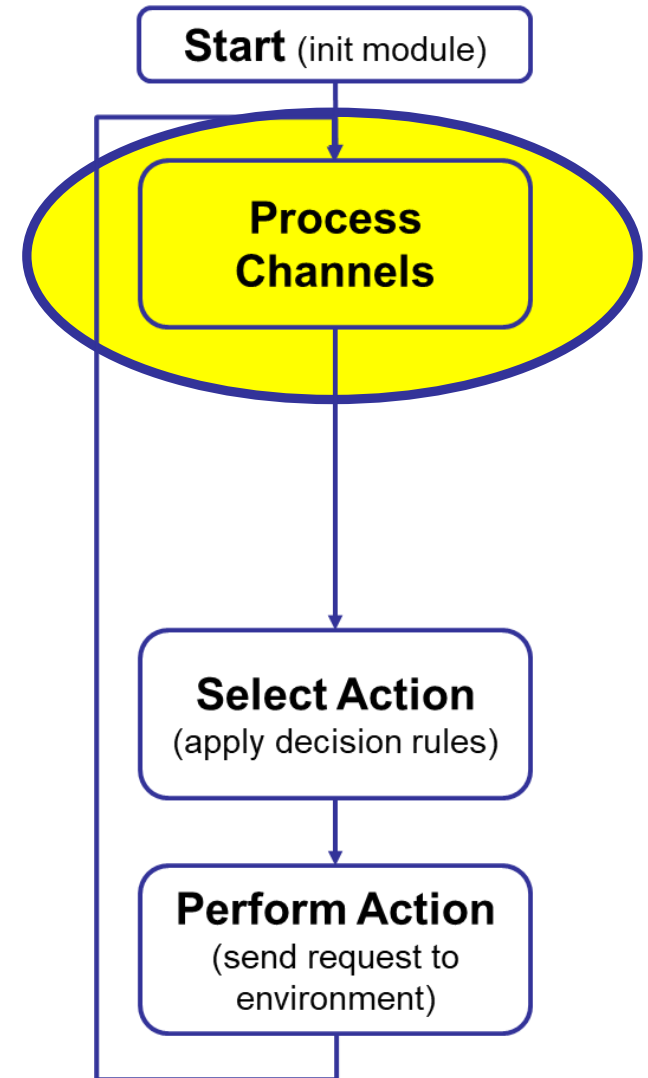
Does agent need to “see” Blocks World?

Can we unsubscribe
agent from on/2 channel?

Is there an alternative
for updating the database?

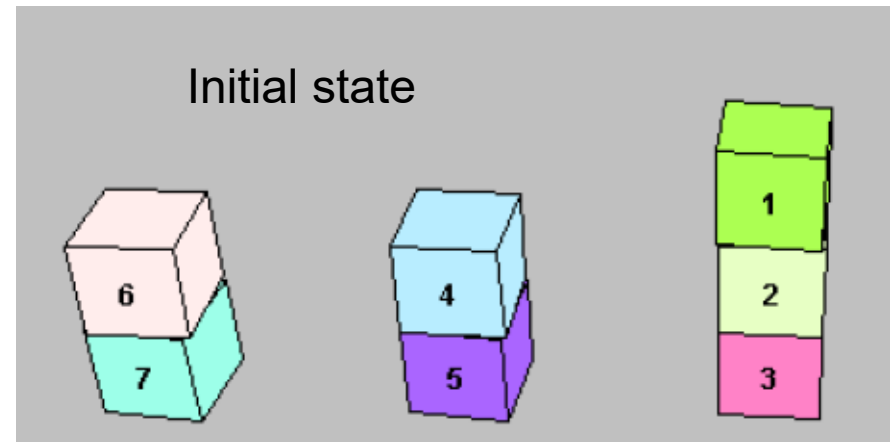
Two things:

- Inserting initial state in DB
- Updating state after move



Initializing the initial state

We can insert the initial state into the database using the `init` module.



```
module stackBuilderInit {  
  if true  
    then insert(agenda([b5,table,table,b3,b2,b4,table])).  
  if true  
    then insert( on(b1,b2), ..., on(b7,table) ).  
}
```

Updating the database with effects

Use insert and delete actions to code action effects; assuming that agent knows X was on Z :

```
move( $X$ ,  $Y$ )  
+ delete(on( $X$ ,  $Z$ )) + insert(on( $X$ ,  $Y$ ))
```

And we can remove the percept channel → the agent does not have to “see” what happens anymore!

Which BW properties make use channel optional?

- Blocks World is deterministic, static, & single agent.
- Using percepts in BW (only) simplifies program.

Organisation

- **Read** MARBEL Programming Guide [Chapter 3](#).
- Thursday 27-11
 - Assignment 3
 - Practical
 - Lecture 7 MARBEL